Bikkad IT
Dream Big! Start Small!!! Act Now!!!

Bikkad IT Institute

Session 4: Python Lists

Practice Questions with Python Code Answers
Compiled by Bikkad IT Institute | Teaching Notes

WHAT IS INSIDE

1. Creating a List
2. Accessing Items from a List
3. Adding Items to a List
4. Editing Items in a List
5. Deleting Items in a List
6. Operators and Loops on a List
7. List Functions
8. List Comprehension
9. Ways to Reverse a List
10. zip() Function
11. Nested Lists & Lists of Any Object

66 practice questions | every topic: 2 Beginner + 2 Intermediate + 2 Advanced

BEGINNER INTERMEDIATE ADVANCED

OUR LOCATIONS

+91 77220 82225 Pimpri-Chinchwad
+91 77220 82226 Chh. Sambhajinagar
+91 77220 82227 Nashik
+91 77220 82224 Hadapsar, Pune

Session 4: Python Lists - Practice Solutions

Prepared for: Bikkad IT Institute Students
Topics: Creating a List, Accessing Items from a List, Adding Items to a List, Editing Items in a List, Deleting Items in a List, Operators and Loops on a List, List Functions, List Comprehension, Ways to Reverse a List, zip() Function, Nested Lists & Lists of Any Object
Levels: 2 Beginner + 2 Intermediate + 2 Advanced questions per topic
Date: October 2026

1. Creating a List

Q1. Weekly Expenses

BEGINNER

Create a list of daily expenses for a week: 250, 180, 320, 150, 400, 600, 220. Print the list and its type.

Solution:

```
expenses = [250, 180, 320, 150, 400, 600, 220]

print(expenses)
print(type(expenses))
```

Output:

```
[250, 180, 320, 150, 400, 600, 220]
<class 'list'>
```

Q2. Empty Cart

BEGINNER

A new customer opens a shopping app. Create an empty cart list and print it and its length.

Solution:

```
cart = []

print(cart)
print(len(cart))
```

Output:

```
[]
0
```

Q3. List from a String and a range

INTERMEDIATE

Convert the word "PUNE" into a list of characters using list(), and create a list of roll numbers 1 to 10 using list(range()).

Solution:

```
letters = list("PUNE")
print(letters)

roll_numbers = list(range(1, 11))
print(roll_numbers)
```

Output:

```
['P', 'U', 'N', 'E']
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Q4. Mixed Product Record

INTERMEDIATE

Create a single list for a product: name "Rice 5kg", price 425.5, quantity 12, in_stock True and tags ["grocery", "daily"]. Print the list and the type of each element using a loop.

Solution:

```
product = ["Rice 5kg", 425.5, 12, True, ["grocery", "daily"]]
print(product)

for item in product:
    print(type(item))
```

Output:

```
['Rice 5kg', 425.5, 12, True, ['grocery', 'daily']]
<class 'str'>
<class 'float'>
<class 'int'>
<class 'bool'>
<class 'list'>
```

Q5. Bus Departure Minutes

ADVANCED

A city bus leaves every 15 minutes from minute 0 to 120. Create the list of departure minutes using list(range()) with a step. Also create a list of the first 8 multiples of 25 using a loop and append().

Solution:

```
departures = list(range(0, 121, 15))
print(departures)

multiples = []
for i in range(1, 9):
    multiples.append(i * 25)
print(multiples)
```

Output:

```
[0, 15, 30, 45, 60, 75, 90, 105, 120]
[25, 50, 75, 100, 125, 150, 175, 200]
```

Q6. Alias vs Real Copy

ADVANCED

Create team_a = ["Om", "Isha", "Raj"]. Make team_b = team_a and team_c = team_a.copy(). Add "Vivek" to team_b and see which lists change. Prove with id().

Solution:

```
team_a = ["Om", "Isha", "Raj"]
team_b = team_a
team_c = team_a.copy()

team_b.append("Vivek")

print(team_a)
print(team_b)
print(team_c)
print(id(team_a) == id(team_b))
print(id(team_a) == id(team_c))
```

Output:

```
['Om', 'Isha', 'Raj', 'Vivek']
['Om', 'Isha', 'Raj', 'Vivek']
['Om', 'Isha', 'Raj']
True
False
```

2. Accessing Items from a List

Q7. First and Last in Queue

BEGINNER

A hospital queue is ["Asha", "Bilal", "Chetan", "Diya", "Eshan"]. Print the first and last patient.

Solution:

```
queue = ["Asha", "Bilal", "Chetan", "Diya", "Eshan"]

print(queue[0])
print(queue[-1])
```

Output:

```
Asha
Eshan
```

Q8. Top 3 Rank Holders

BEGINNER

Students in rank order are ["Gauri", "Harsh", "Irfan", "Jaya", "Kunal", "Lata"]. Print the top 3 using slicing.

Solution:

```
ranks = ["Gauri", "Harsh", "Irfan", "Jaya", "Kunal", "Lata"]

print(ranks[:3])
```

Output:

```
['Gauri', 'Harsh', 'Irfan']
```

Q9. Last 3 Transactions

INTERMEDIATE

Bank transactions are [1200, -500, 3000, -750, 450, -200]. Print the last 3 transactions using negative slicing and the 2nd last transaction.

Solution:

```
transactions = [1200, -500, 3000, -750, 450, -200]

print(transactions[-3:])
print(transactions[-2])
```

Output:

```
[-750, 450, -200]
450
```

Q10. Cinema Seat from Nested List

INTERMEDIATE

Cinema seats are stored row-wise: [["A1", "A2", "A3"], ["B1", "B2", "B3"], ["C1", "C2", "C3"]]. Print the complete second row and the seat "C2".

Solution:

```
seats = [["A1", "A2", "A3"],
         ["B1", "B2", "B3"],
         ["C1", "C2", "C3"]]

print(seats[1])
print(seats[2][1])
```

Output:

```
['B1', 'B2', 'B3']
C2
```

Q11. Middle, Alternate and Reverse

ADVANCED

For the monthly sales [45, 52, 38, 60, 71, 49, 55]: print the middle value, every alternate month starting from the first, and the list in reverse using slicing.

Solution:

```
sales = [45, 52, 38, 60, 71, 49, 55]

middle = sales[len(sales) // 2]
print(middle)

print(sales[::2])
print(sales[::-1])
```

Output:

```
60
[45, 38, 71, 55]
[55, 49, 71, 60, 38, 52, 45]
```

Q12. Safe Search with index()

ADVANCED

In the product list ["Pen", "Pencil", "Eraser", "Scale"] find the position of "Eraser". Then search for "Marker" safely: first check with in, and only then call index().

Solution:

```
items = ["Pen", "Pencil", "Eraser", "Scale"]

print(items.index("Eraser"))

search = "Marker"
if search in items:
    print(items.index(search))
else:
    print("Not available")
```

Output:

```
2
Not available
```

3. Adding Items to a List

Q13. New Online Order

BEGINNER

Order list is ["ORD101", "ORD102"]. A new order "ORD103" arrives. Add it at the end using append().

Solution:

```
orders = ["ORD101", "ORD102"]

orders.append("ORD103")
print(orders)
```

Output:

```
['ORD101', 'ORD102', 'ORD103']
```

Q14. VIP Customer First

BEGINNER

A queue is ["Rina", "Sahil", "Tara"]. A VIP customer "Mr. Mehta" must be served first. Insert him at index 0.

Solution:

```
queue = ["Rina", "Sahil", "Tara"]

queue.insert(0, "Mr. Mehta")
print(queue)
```

Output:

```
['Mr. Mehta', 'Rina', 'Sahil', 'Tara']
```

Q15. Merge Two Carts

INTERMEDIATE

A user's mobile cart is ["Charger", "Cover"] and laptop cart is ["Mouse", "Keyboard"]. Add all laptop cart items into the mobile cart using extend().

Solution:

```
mobile_cart = ["Charger", "Cover"]
laptop_cart = ["Mouse", "Keyboard"]

mobile_cart.extend(laptop_cart)
print(mobile_cart)
```

Output:

```
['Charger', 'Cover', 'Mouse', 'Keyboard']
```

Q16. append() vs extend()

INTERMEDIATE

Start with two lists [10, 20]. Add [30, 40] to the first using append() and to the second using extend(). Print both and their lengths.

Solution:

```
list1 = [10, 20]
list2 = [10, 20]

list1.append([30, 40])
list2.extend([30, 40])

print(list1)
print(len(list1))
print(list2)
print(len(list2))
```

Output:

```
[10, 20, [30, 40]]
3
[10, 20, 30, 40]
4
```

Q17. Collect Inputs Until done

ADVANCED

Keep taking expense amounts from the user and append them to a list until the user types "done". Then print the list and total expense.

Solution:

```
expenses = []

while True:
    value = input("Enter expense (or done): ")
    if value == "done":
        break
    expenses.append(int(value))

print(expenses)
print(sum(expenses))
```

Output (with sample input):

```
Enter expense (or done): 450
Enter expense (or done): 120
Enter expense (or done): 980
Enter expense (or done): done
[450, 120, 980]
1550
```

Q18. Merge Two Sorted Lists

ADVANCED

Two branches have sorted student IDs [3, 8, 15, 22] and [5, 9, 18]. Merge them into one sorted list WITHOUT using sort(), by comparing items one by one.

Solution:

```
a = [3, 8, 15, 22]
b = [5, 9, 18]
merged = []
i = 0
j = 0

while i < len(a) and j < len(b):
    if a[i] <= b[j]:
        merged.append(a[i])
        i += 1
    else:
        merged.append(b[j])
        j += 1

merged.extend(a[i:])
merged.extend(b[j:])
print(merged)
```

Output:

```
[3, 5, 8, 9, 15, 18, 22]
```

4. Editing Items in a List

Q19. Update a Price

BEGINNER

Prices are [120, 340, 560, 90]. The second product's price changes to 360. Update it using its index.

Solution:

```
prices = [120, 340, 560, 90]

prices[1] = 360
print(prices)
```

Output:

```
[120, 360, 560, 90]
```

Q20. Change the Last Stop

BEGINNER

A bus route is ["Swargate", "Deccan", "Shivajinagar", "Aundh"]. The last stop changes to "Baner". Update it using index -1.

Solution:

```
route = ["Swargate", "Deccan", "Shivajinagar", "Aundh"]

route[-1] = "Baner"
print(route)
```

Output:

```
['Swargate', 'Deccan', 'Shivajinagar', 'Baner']
```

Q21. Replace Several Items with Slicing

INTERMEDIATE

Weekly scores are [50, 60, 70, 80, 90]. Scores at positions 1 to 3 were wrong; replace them with [65, 75, 85] using slice assignment.

Solution:

```
scores = [50, 60, 70, 80, 90]

scores[1:4] = [65, 75, 85]
print(scores)
```

Output:

```
[50, 65, 75, 85, 90]
```

Q22. 10% Price Hike

INTERMEDIATE

Increase every price in [200, 450, 800, 1200] by 10% using a loop with index (update the same list).

Solution:

```
prices = [200, 450, 800, 1200]

for i in range(len(prices)):
    prices[i] = prices[i] * 1.10
    prices[i] = round(prices[i], 2)

print(prices)
```

Output:

```
[220.0, 495.0, 880.0, 1320.0]
```

Q23. Replace All Occurrences

ADVANCED

In the attendance list ["P", "A", "P", "L", "A", "P"], replace every "A" (absent) with "Absent" and every "L" with "Leave".

Solution:

```
attendance = ["P", "A", "P", "L", "A", "P"]

for i in range(len(attendance)):
    if attendance[i] == "A":
        attendance[i] = "Absent"
    elif attendance[i] == "L":
        attendance[i] = "Leave"

print(attendance)
```

Output:

```
['P', 'Absent', 'P', 'Leave', 'Absent', 'P']
```

Q24. Insert Using Slice and Capitalize In Place

ADVANCED

Students list is ["ajit", "bhavna", "farah"]. Insert ["chirag", "deepa"] at index 2 using an empty slice (students[2:2] = ...), then capitalize every name in place.

Solution:

```
students = ["ajit", "bhavna", "farah"]

students[2:2] = ["chirag", "deepa"]
print(students)

for i in range(len(students)):
    students[i] = students[i].capitalize()

print(students)
```

Output:

```
['ajit', 'bhavna', 'chirag', 'deepa', 'farah']
['Ajit', 'Bhavna', 'Chirag', 'Deepa', 'Farah']
```

5. Deleting Items in a List

Q25. Cancel an Order

BEGINNER

Orders are ["Pizza", "Burger", "Pasta", "Fries"]. The customer cancels "Pasta". Remove it by value.

Solution:

```
orders = ["Pizza", "Burger", "Pasta", "Fries"]

orders.remove("Pasta")
print(orders)
```

Output:

```
['Pizza', 'Burger', 'Fries']
```

Q26. Read the Latest Notification

BEGINNER

Notifications are ["OTP sent", "Order shipped", "Payment done"]. Remove and print the latest (last) notification using pop().

Solution:

```
notifications = ["OTP sent", "Order shipped", "Payment done"]

latest = notifications.pop()
print(latest)
print(notifications)
```

Output:

```
Payment done
['OTP sent', 'Order shipped']
```

Q27. Delete by Index and Slice

INTERMEDIATE

Playlist is ["Song1", "Song2", "Song3", "Song4", "Song5", "Song6"]. Delete the 2nd song using del, then delete the last two songs using a slice.

Solution:

```
playlist = ["Song1", "Song2", "Song3", "Song4", "Song5", "Song6"]

del playlist[1]
print(playlist)

del playlist[-2:]
print(playlist)
```

Output:

```
['Song1', 'Song3', 'Song4', 'Song5', 'Song6']
['Song1', 'Song3', 'Song4']
```

Q28. Empty the Cart After Payment

INTERMEDIATE

After successful payment, clear the cart ["Shoes", "Socks", "Belt"] using clear() and print the list and its length.

Solution:

```
cart = ["Shoes", "Socks", "Belt"]

cart.clear()
print(cart)
print(len(cart))
```

Output:

```
[]
0
```

Q29. Remove All Occurrences of a Value

ADVANCED

remove() deletes only the first match. Remove ALL "spam" entries from ["hi", "spam", "hello", "spam", "bye", "spam"] using a while loop.

Solution:

```
messages = ["hi", "spam", "hello", "spam", "bye", "spam"]

while "spam" in messages:
    messages.remove("spam")

print(messages)
```

Output:

```
['hi', 'hello', 'bye']
```

Q30. Remove Duplicates, Keep Order

ADVANCED

Website visitors list is ["Pune", "Delhi", "Pune", "Goa", "Delhi", "Pune"]. Remove duplicates but keep the original order (do not use set).

Solution:

```
visitors = ["Pune", "Delhi", "Pune", "Goa", "Delhi", "Pune"]
unique = []

for city in visitors:
    if city not in unique:
        unique.append(city)

print(unique)
```

Output:

```
['Pune', 'Delhi', 'Goa']
```

6. Operators and Loops on a List

Q31. Combine Two Batches

BEGINNER

Morning batch is ["Akash", "Bina"] and evening batch is ["Chirag", "Dolly"]. Combine them using +.

Solution:

```
morning = ["Akash", "Bina"]
evening = ["Chirag", "Dolly"]

all_students = morning + evening
print(all_students)
```

Output:

```
['Akash', 'Bina', 'Chirag', 'Dolly']
```

Q32. Weekly Counter

BEGINNER

Create a list of 7 zeros to count daily website visits using the * operator.

Solution:

```
visits = [0] * 7

print(visits)
print(len(visits))
```

Output:

```
[0, 0, 0, 0, 0, 0, 0]
7
```

Q33. Check Registration

INTERMEDIATE

Registered students are ["Neel", "Ojas", "Prachi"]. Check whether "Prachi" is registered and whether "Rutuja" is NOT registered.

Solution:

```
registered = ["Neel", "Ojas", "Prachi"]

print("Prachi" in registered)
print("Rutuja" not in registered)
```

Output:

```
True
True
```

Q34. Leaderboard with enumerate()

INTERMEDIATE

Print a leaderboard for ["Shreya", "Tejas", "Uday"] with rank numbers starting from 1 using enumerate().

Solution:

```
players = ["Shreya", "Tejas", "Uday"]

for rank, name in enumerate(players, start=1):
    print(rank, name)
```

Output:

```
1 Shreya
2 Tejas
3 Uday
```

Q35. Serve a Queue Until Empty

ADVANCED

A ticket counter queue is ["T1", "T2", "T3", "T4"]. Use a while loop to serve (pop from the front) until the queue is empty, printing who is served.

Solution:

```
queue = ["T1", "T2", "T3", "T4"]

while queue:
    served = queue.pop(0)
    print(served)

print(queue)
```

Output:

```
T1
T2
T3
T4
[]
```

Q36. Above-Average Students

ADVANCED

Marks are [67, 82, 45, 90, 73, 58]. Calculate the average using a for loop (no sum()) and print the marks that are above average.

Solution:

```
marks = [67, 82, 45, 90, 73, 58]
total = 0

for m in marks:
    total += m

average = total / len(marks)
print(round(average, 2))

for m in marks:
    if m > average:
        print(m)
```

Output:

```
69.17
82
90
73
```

7. List Functions

Q37. Temperature Summary

BEGINNER

A week's temperatures are [31, 34, 29, 36, 33, 30, 35]. Print the count of days, the minimum and the maximum temperature.

Solution:

```
temps = [31, 34, 29, 36, 33, 30, 35]

print(len(temps))
print(min(temps))
print(max(temps))
```

Output:

```
7
29
36
```

Q38. Total and Average Sales

BEGINNER

Daily sales are [4500, 6200, 3800, 7100, 5400]. Print the total and the average sales.

Solution:

```
sales = [4500, 6200, 3800, 7100, 5400]

total = sum(sales)
print(total)

average = total / len(sales)
print(average)
```

Output:

```
27000
5400.0
```

Q39. Sort Prices

INTERMEDIATE

Prices are [799, 299, 1499, 99, 549]. Sort them in ascending order using sort(), then in descending order.

Solution:

```
prices = [799, 299, 1499, 99, 549]

prices.sort()
print(prices)

prices.sort(reverse=True)
print(prices)
```

Output:

```
[99, 299, 549, 799, 1499]
[1499, 799, 549, 299, 99]
```

Q40. sorted() with key and count()

INTERMEDIATE

Names are ["Om", "Siddharth", "Riya", "Ganesh", "Om"]. Print a new list sorted by name length using sorted(key=len) (original unchanged) and count how many times "Om" appears.

Solution:

```
names = ["Om", "Siddharth", "Riya", "Ganesh", "Om"]

by_length = sorted(names, key=len)
print(by_length)
print(names)

print(names.count("Om"))
```

Output:

```
['Om', 'Om', 'Riya', 'Ganesh', 'Siddharth']
['Om', 'Siddharth', 'Riya', 'Ganesh', 'Om']
2
```

Q41. Second Highest Score

ADVANCED

Scores are [88, 95, 72, 95, 81, 88]. Find the second highest UNIQUE score.

Solution:

```
scores = [88, 95, 72, 95, 81, 88]

unique_scores = list(set(scores))
unique_scores.sort(reverse=True)
print(unique_scores)

second_highest = unique_scores[1]
print(second_highest)
```

Output:

```
[95, 88, 81, 72]
88
```

Q42. Frequency of Each Item

ADVANCED

In a class monitor election the votes are ["A", "B", "C", "A", "C", "A"] (A, B, C are candidates). Count the frequency of each option using count() and print the winner using max() with key.

Solution:

```
votes = ["A", "B", "C", "A", "C", "A"]
options = sorted(set(votes))

for opt in options:
    print(opt, votes.count(opt))

winner = max(options, key=votes.count)
print(winner)
```

Output:

```
A 3
B 1
C 2
A
```

8. List Comprehension

Q43. Squares of 1 to 10

BEGINNER

Create a list of squares of numbers from 1 to 10 using list comprehension.

Solution:

```
squares = [n ** 2 for n in range(1, 11)]
print(squares)
```

Output:

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Q44. Product Names in Uppercase

BEGINNER

Convert ["soap", "shampoo", "toothpaste"] to uppercase using list comprehension.

Solution:

```
products = ["soap", "shampoo", "toothpaste"]

upper_products = [p.upper() for p in products]
print(upper_products)
```

Output:

```
['SOAP', 'SHAMPOO', 'TOOTHPASTE']
```

Q45. Passed Students Only

INTERMEDIATE

Marks are [34, 67, 89, 21, 45, 76, 33]. Using list comprehension with a condition, keep only the marks >= 35.

Solution:

```
marks = [34, 67, 89, 21, 45, 76, 33]

passed = [m for m in marks if m >= 35]
print(passed)
```

Output:

```
[67, 89, 45, 76]
```

Q46. Prices with GST

INTERMEDIATE

Base prices are [100, 250, 499, 1200]. Create a new list with 18% GST added and rounded to 2 decimals.

Solution:

```
prices = [100, 250, 499, 1200]

with_gst = [round(p * 1.18, 2) for p in prices]
print(with_gst)
```

Output:

```
[118.0, 295.0, 588.82, 1416.0]
```

Q47. Pass/Fail Labels (if-else inside)

ADVANCED

For marks [34, 67, 89, 21, 45], create a list of labels "Pass" or "Fail" using if-else inside the comprehension. Also create (mark, label) pairs.

Solution:

```
marks = [34, 67, 89, 21, 45]

labels = ["Pass" if m >= 35 else "Fail" for m in marks]
print(labels)

pairs = [(m, "Pass" if m >= 35 else "Fail") for m in marks]
print(pairs)
```

Output:

```
['Fail', 'Pass', 'Pass', 'Fail', 'Pass']
[(34, 'Fail'), (67, 'Pass'), (89, 'Pass'), (21, 'Fail'), (45,
'Pass')]
```

Q48. Flatten and Transpose

ADVANCED

Branch-wise sales are [[10, 20, 30], [40, 50, 60]]. Flatten it into one list using a nested comprehension and transpose it (rows become columns).

Solution:

```
sales = [[10, 20, 30],
         [40, 50, 60]]

flat = [value for row in sales for value in row]
print(flat)

columns = len(sales[0])
transpose = [[row[c] for row in sales] for c in range(columns)]
print(transpose)
```

Output:

```
[10, 20, 30, 40, 50, 60]
[[10, 40], [20, 50], [30, 60]]
```

9. Ways to Reverse a List

Q49. Latest Orders First

BEGINNER

Order history is ["Jan", "Feb", "Mar", "Apr"]. Show the latest first using reverse().

Solution:

```
history = ["Jan", "Feb", "Mar", "Apr"]

history.reverse()
print(history)
```

Output:

```
['Apr', 'Mar', 'Feb', 'Jan']
```

Q50. Reverse Using Slicing

BEGINNER

Reverse the list of floors [1, 2, 3, 4, 5] for a lift going down using slicing, keeping the original list unchanged.

Solution:

```
floors = [1, 2, 3, 4, 5]

going_down = floors[::-1]
print(going_down)
print(floors)
```

Output:

```
[5, 4, 3, 2, 1]
[1, 2, 3, 4, 5]
```

Q51. Countdown with reversed()

INTERMEDIATE

Print a countdown of seconds [10, 20, 30, 40] from last to first using reversed() in a for loop, and also convert reversed() into a list.

Solution:

```
seconds = [10, 20, 30, 40]

for s in reversed(seconds):
    print(s)

print(list(reversed(seconds)))
```

Output:

```
40
30
20
10
[40, 30, 20, 10]
```

Q52. Reverse Using insert(0)

INTERMEDIATE

Reverse ["A", "B", "C", "D"] into a new list by inserting each item at index 0.

Solution:

```
letters = ["A", "B", "C", "D"]
result = []

for ch in letters:
    result.insert(0, ch)

print(result)
```

Output:

```
['D', 'C', 'B', 'A']
```

Q53. Two-Pointer Swap

ADVANCED

Reverse [11, 22, 33, 44, 55, 66] in place using two pointers (left and right) and a while loop - without reverse() or slicing.

Solution:

```
nums = [11, 22, 33, 44, 55, 66]
left = 0
right = len(nums) - 1

while left < right:
    nums[left], nums[right] = nums[right], nums[left]
    left += 1
    right -= 1

print(nums)
```

Output:

```
[66, 55, 44, 33, 22, 11]
```

Q54. Reverse Words and Their Order

ADVANCED

For ["pune", "goa", "delhi"]: (1) reverse the spelling of each word, (2) reverse the order of the list, and (3) do both together.

Solution:

```
cities = ["pune", "goa", "delhi"]

each_word = [c[::-1] for c in cities]
print(each_word)

order = cities[::-1]
print(order)

both = [c[::-1] for c in cities[::-1]]
print(both)
```

Output:

```
['enup', 'aog', 'ihled']
['delhi', 'goa', 'pune']
['ihled', 'aog', 'enup']
```

10. zip() Function

Q55. Student and Marks Pairs

BEGINNER

Names are ["Asha", "Bhushan", "Chaitra"] and marks are [78, 85, 92]. Use zip() to print each name with its marks.

Solution:

```
names = ["Asha", "Bhushan", "Chaitra"]
marks = [78, 85, 92]

for n, m in zip(names, marks):
    print(n, m)
```

Output:

```
Asha 78
Bhushan 85
Chaitra 92
```

Q56. Price Dictionary

BEGINNER

Products are ["Tea", "Coffee", "Juice"] and prices are [15, 25, 40]. Create a dictionary using dict(zip()).

Solution:

```
products = ["Tea", "Coffee", "Juice"]
prices = [15, 25, 40]

menu = dict(zip(products, prices))
print(menu)
```

Output:

```
{'Tea': 15, 'Coffee': 25, 'Juice': 40}
```

Q57. Three-List Report

INTERMEDIATE

Combine employees ["Ketan", "Leena"], departments ["IT", "HR"] and salaries [52000, 47000] using zip() and print a formatted line for each.

Solution:

```
employees = ["Ketan", "Leena"]
departments = ["IT", "HR"]
salaries = [52000, 47000]

for e, d, s in zip(employees, departments, salaries):
    print(f"{e} works in {d} and earns {s}")
```

Output:

```
Ketan works in IT and earns 52000
Leena works in HR and earns 47000
```

Q58. Unequal Lengths

INTERMEDIATE

Days are ["Mon", "Tue", "Wed", "Thu"] but sales are recorded only for 3 days [500, 700, 650]. Zip them and observe that zip stops at the shortest list.

Solution:

```
days = ["Mon", "Tue", "Wed", "Thu"]
sales = [500, 700, 650]

paired = list(zip(days, sales))
print(paired)
print(len(paired))
```

Output:

```
[('Mon', 500), ('Tue', 700), ('Wed', 650)]
3
```

Q59. Unzip Records

ADVANCED

Records are [("Pen", 10), ("Book", 60), ("Bag", 450)]. Separate them into an items tuple and a prices tuple using zip(*records), then print the total price.

Solution:

```
records = [("Pen", 10), ("Book", 60), ("Bag", 450)]

items, prices = zip(*records)
print(items)
print(prices)
print(sum(prices))
```

Output:

```
('Pen', 'Book', 'Bag')
(10, 60, 450)
520
```

Q60. Auto-Grade an MCQ Test

ADVANCED

Answer key is ["B", "D", "A", "C", "B"] and a student's answers are ["B", "C", "A", "C", "D"]. Use zip() to compare answers, count the score and list the question numbers that were wrong.

Solution:

```
key = ["B", "D", "A", "C", "B"]
student = ["B", "C", "A", "C", "D"]
score = 0
wrong = []

for q, (k, s) in enumerate(zip(key, student), start=1):
    if k == s:
        score += 1
    else:
        wrong.append(q)

print(score)
print(wrong)
```

Output:

```
3
[2, 5]
```

11. Nested Lists & Lists of Any Object

Q61. Marks Table

BEGINNER

Marks of 3 students in 3 subjects are [[78, 85, 90], [66, 72, 80], [88, 91, 79]]. Print the second student's marks and the third subject mark of the first student.

Solution:

```
marks = [[78, 85, 90],
         [66, 72, 80],
         [88, 91, 79]]

print(marks[1])
print(marks[0][2])
```

Output:

```
[66, 72, 80]
90
```

Q62. List of Dictionaries

BEGINNER

Store two books as dictionaries inside a list: {"title": "Wings of Fire", "price": 299} and {"title": "Python Basics", "price": 450}. Print the title of the second book.

Solution:

```
books = [
    {"title": "Wings of Fire", "price": 299},
    {"title": "Python Basics", "price": 450}
]

print(books[1]["title"])
```

Output:

```
Python Basics
```

Q63. Row-wise Totals

INTERMEDIATE

Quarterly sales of 3 branches are [[120, 150, 130, 160], [90, 110, 100, 95], [200, 180, 210, 220]]. Print the total sales of each branch.

Solution:

```
sales = [[120, 150, 130, 160],
         [90, 110, 100, 95],
         [200, 180, 210, 220]]

for branch in sales:
    print(sum(branch))
```

Output:

```
560
395
810
```

Q64. Column Total

INTERMEDIATE

Using the same quarterly sales list, find the total sales of Quarter 2 (column index 1) for all branches using a loop.

Solution:

```
sales = [[120, 150, 130, 160],
         [90, 110, 100, 95],
         [200, 180, 210, 220]]

q2_total = 0
for branch in sales:
    q2_total += branch[1]

print(q2_total)
```

Output:

```
440
```

Q65. Filter Employees by Salary

ADVANCED

Employees are stored as a list of dictionaries with name, dept and salary. Print the names of employees earning more than 50000 and the total salary of the IT department.

Solution:

```
employees = [
    {"name": "Nitin", "dept": "IT", "salary": 62000},
    {"name": "Pallavi", "dept": "HR", "salary": 48000},
    {"name": "Qasim", "dept": "IT", "salary": 55000},
    {"name": "Rekha", "dept": "Sales", "salary": 51000}
]

for e in employees:
    if e["salary"] > 50000:
        print(e["name"])

it_total = 0
for e in employees:
    if e["dept"] == "IT":
        it_total += e["salary"]
print(it_total)
```

Output:

```
Nitin
Qasim
Rekha
117000
```

Q66. Add Two Stock Matrices

ADVANCED

Stock of 2 shops for 2 products in 2 weeks: week1 = [[10, 20], [30, 40]] and week2 = [[5, 15], [25, 35]]. Add them element-wise using nested loops to get the total stock matrix.

Solution:

```
week1 = [[10, 20], [30, 40]]
week2 = [[5, 15], [25, 35]]
total = []

for i in range(len(week1)):
    row = []
    for j in range(len(week1[0])):
        row.append(week1[i][j] + week2[i][j])
    total.append(row)

print(total)
```

Output:

```
[[15, 35], [55, 75]]
```

End of Solutions

Prepared by: Bikkad IT Institute
For: Python Beginner Level Students
Date: October 2026